MOBILE MAPPING SYSTEM · PROCESSING PLATFORM

Production Setup Guide

End-to-end installation, configuration and verification
for the GeoSphere 360 WebGIS dashboard and on-premises survey pipeline

DOCUMENT	GeoSphere 360 — Production Setup Guide
REVISION	1.0
APPLIES TO	geosphere-360-dashboard
AUDIENCE	Client IT / infrastructure team, and client survey operations leads
SCOPE	Cloud deployment, database, on-premises services, configuration, verification
STATUS	For implementation

Read this first. This platform performs **no server-side image processing** and requires **no GPU**. Panorama quality analysis runs in the operator's browser on a WebGL shader path; image editing is performed by operators in desktop software on four workstation PCs, which this platform observes. Do not procure GPU hardware or CUDA toolchains for this system. See **Section 1.2**.

AT A GLANCE

GPU REQUIRED **No**	TUNNEL HOSTNAMES **5**	DATABASE MIGRATIONS **33 files**
ON-PREM SERVICES **5**	VERIFICATION TIERS **2**	HOSTING TARGET **Cloudflare Pages**

Known limitations are catalogued in **Section 14**. Two require a client decision before go-live: panorama bucket privacy (4.1) and database-enforced deletion approval (6.3).

Proprietary & confidential. Unauthorised distribution is prohibited.

How to read this guide, and what it assumes.

Purpose

This guide takes a client from an empty account to a verified, running GeoSphere 360 installation. It is written to be followed in order, by someone who has not seen the system before. Every step states what to run, the expected result, and what to do when it differs.

How the guide is organised

The work divides into four phases. Each phase can be handed to a different engineer, and later phases depend on earlier ones.

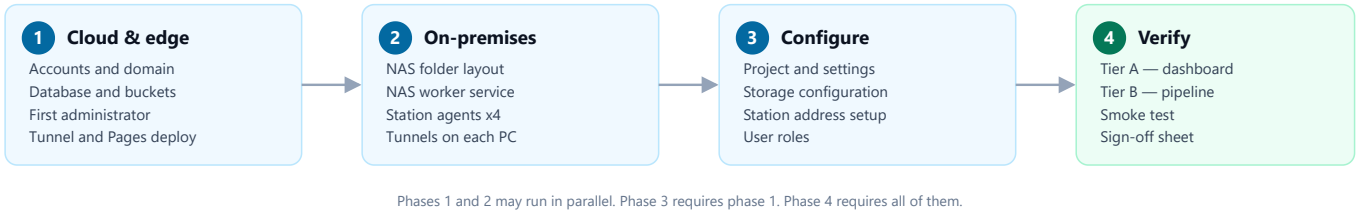


Figure 1. The four install phases and their dependencies.

Conventions

CONVENTION	MEANING
`<ANGLE_BRACKETS>`	A placeholder. Replace the text *and* the brackets.
`VITE_SOMETHING`	An environment variable name. Names are case-sensitive.
Windows / **Linux**	PowerShell versus bash. Commands are otherwise identical.
Section *n*	A numbered section of this guide.
Blue callout	Information you will need.
Amber callout	A common mistake. Check here when something does not match.
Red callout	A known limitation or blocker. Read before go-live.

Build-time versus runtime configuration

Variables prefixed `VITE_` are compiled into the browser bundle at build time. Adding or changing one requires a **rebuild** — editing the variable and restarting the application does nothing. Variables without that prefix are read server-side by Cloudflare and never reach the browser. Sections 8.3 and 8.4 list which is which.

1 **About this system**
What it is, what it is not, architecture, components

2 **Prerequisites**
Accounts, domain and hostnames, hardware, software versions

3 **Database**
Project creation, PostGIS, the migration sequence, verification

4 **Storage buckets**
Panorama bucket, geometry bucket, privacy implications

5 **First administrator**
Manual bootstrap, role resolution, adding further users

6 **Roles and row-level security**
The four roles, what the database actually enforces

7 **Cloudflare Tunnel**
Named tunnels, configuration, DNS routing, running as a service

8 **Cloudflare Pages deployment**
Build settings, build-time and runtime variables, hosting constraint

9 **NAS layout**Required folder tree, depth rules, subgrid naming

10 **NAS worker**
Install, configure, run, verify

11 **Station agents**
One per workstation, configuration, firewall, autostart

12 **Onboarding a client company**
Seller Console: create the company, invite staff, what the client sees

13 **First-run configuration**
Project, settings, station address setup, roles

14 **Verification**
Tier A and Tier B acceptance, smoke test, sign-off sheet

15 **Known limitations**
Product defects that affect a client installation

16 **Troubleshooting**
Symptom to cause, including silent failures

17 **Reference**
Environment variables, migrations, HTTP surfaces, roles

What GeoSphere 360 is, how it is put together, and what it does not do.

1.1 What it is

GeoSphere 360 is a browser-based WebGIS dashboard for 360 degree mobile-mapping survey data, together with a small set of on-premises services that connect it to a survey NAS and to four photogrammetry workstations.

A surveyor captures 360 imagery and GNSS/IMU telemetry on a vehicle rig. Operator PCs run desktop photogrammetry software to blur, stitch, enhance and retouch the imagery. The dashboard coordinates that work: it shows which subgrid has been captured, what stage each workstation has reached, which frames failed quality inspection, and what has been published to the WebGIS layer.

1.2 What it is not

This distinction drives hardware purchasing, so it is stated plainly and early.

No server-side image processing

The platform does **not** run a server-side image pipeline. There is no GPU worker, no batch enhancement queue, and no image-processing job store.

- **Panorama quality analysis** — sharpness, privacy-blur verification, obstruction and glare detection — runs **in the operator's browser** on a WebGL shader path, with a CPU fallback.
- **Image editing** — blurring, stitching, enhancement, nadir removal — is performed by operators in desktop software on the four workstations. The dashboard observes their progress; it does not drive them.

You do not need: an NVIDIA GPU, CUDA, PyTorch, OpenCV, a GPU workstation, or a GPU queue service. Standard office hardware is sufficient. Section 2.3 gives the actual list.

1.3 Architecture

The system splits into a cloud half, which the client's users reach over HTTPS, and an on-premises half that never touches the public internet. A Cloudflare Tunnel bridges them.

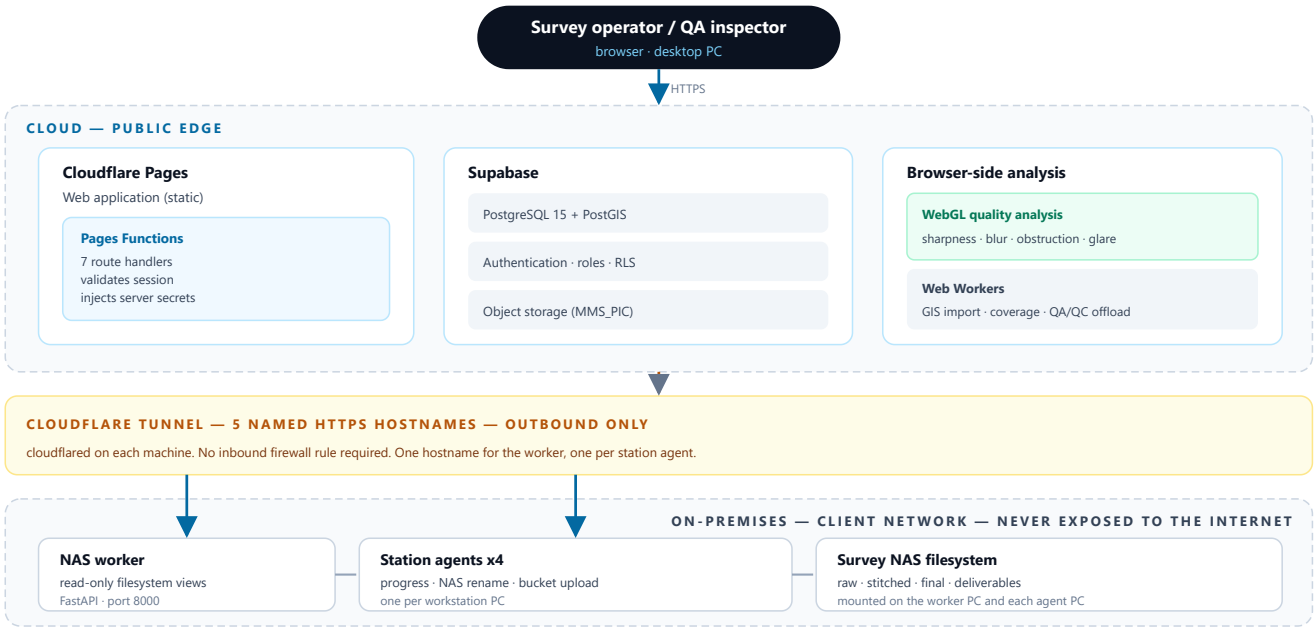


Figure 2. System architecture. The browser never receives an on-premises credential — Pages Functions validate the session and inject server-side secrets before proxying. The NAS is reachable only by the worker and the agents.

COMPONENT	RESPONSIBILITY	AUTHORITATIVE FOR
Cloudflare Pages	Serves the web application; hosts the private-API proxy	—
Pages Functions	Validate the signed-in session, then proxy to on-premises services using server-side credentials	Never exposes an on-premises credential to the browser
Supabase	Spatial database, authentication, object storage	All persistent state
NAS worker	Read-only view of the survey filesystem and storage telemetry	What is on disk
Station agents x4	Report workstation progress; perform NAS rename and bucket upload	Per-workstation work state
Operator browser	Panorama quality analysis on a WebGL path	Quality verdicts

Design principle

The database is the single source of truth. The NAS is treated as read-only by the dashboard: the worker rejects any path that resolves outside its configured base, so a malformed request cannot reach unrelated files on the client network.

Accounts, domain, hostnames, hardware and software — everything to acquire before starting.

2.1 Install order at a glance

Perform these in sequence. Steps 6 to 9 are the on-premises half and may be carried out in parallel with steps 1 to 5 by a different engineer.

#	ACTION	SECTION	ESTIMATE
1	Create Cloudflare and Supabase accounts; attach a domain	2.2, 2.4	1 day
2	Create the Supabase project; **enable PostGIS first**	3.1, 3.2	30 min
3	Apply migrations `0001` – `0010` then `0012` – `0033`	3.3	45 min
4	Create the `MMS_PIC` storage bucket (public)	4.1	5 min
5	Create the first administrator	5	15 min
6	Build the NAS folder tree	9	1 day
7	Install and run the NAS worker	10	30 min
8	Install and run four station agents	11	2 hours
9	Create five Cloudflare Tunnels	7	1 hour
10	Deploy to Cloudflare Pages	8	30 min
11	Configure the application	12	45 min
12	Verify the installation	13	30 min

2.2 Accounts and services

NEED	WHY	NOTES
Cloudflare account	Hosts the web deployment, the tunnels and optional access policies	A free account is sufficient to start
A domain on Cloudflare DNS	The tunnel hostnames must be real, resolvable HTTPS names	Section 2.4 — you need five of them
Supabase project	PostgreSQL and PostGIS, authentication, object storage	Supabase Cloud, or self-hosted Docker
Photogrammetry software licences	Runs on the four workstations	Not part of this platform
A WebGIS application URL	The embedded interactive map	Required at build time; there is no fallback

2.3 Hardware

No GPU is required for any component of this platform (Section 1.2).

NAS / worker PC	4 cores, 8 GB RAM, 100 GB free disk	8 cores, 16 GB RAM, SSD
Each station PC (x4)	4 cores, 8 GB RAM, SSD	8 cores, 16 GB RAM, SSD
Network	1 GbE between the PCs and the NAS	10 GbE for large surveys
NAS capacity	Raw + stitched + final + deliverables	Plan roughly three times the raw survey volume
Cloud	Managed Supabase, free Cloudflare tier	Scale the database for archive volumes

> **Size the workstations for the photogrammetry software**
>
> The station PCs also run stitching and image-editing software, which is far more demanding than this platform. Choose their specification for that workload, not for the dashboard.

2.4 Hostnames — plan for five

The dashboard reaches the on-premises services through five separate tunnels. This is the single most commonly under-provisioned item, so plan for it before ordering certificates or DNS.

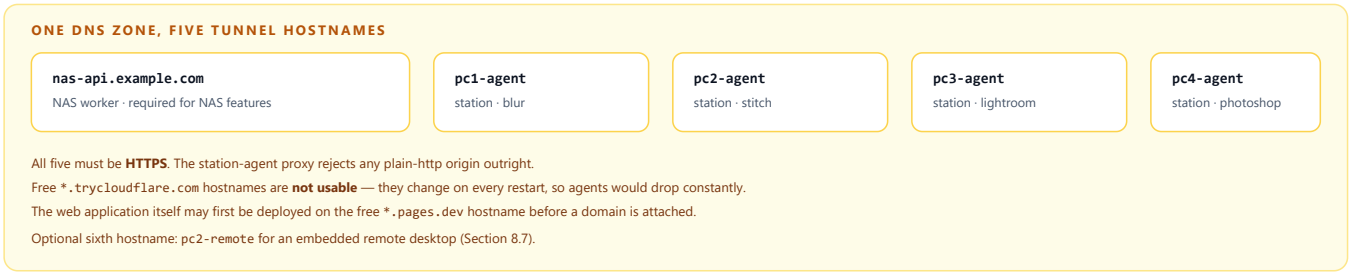


Figure 3. Required hostname allocation.

2.5 Software versions

COMPONENT	VERSION	NOTES
Node.js	18.x or 20.x LTS	Only needed on the machine that builds the application
npm	9.0 or higher	Ships with Node 18 and 20
Python	3.10 or higher	Runs the NAS worker and the station agents
cloudflared	current	Runs on the worker PC and each station PC
PostgreSQL	15 with PostGIS 3.3	Provided by Supabase

Project creation, PostGIS, the migration sequence and verification.

3.1 Create the project

Supabase Cloud — create a project in your organisation. **Self-hosted** — run the Supabase Docker stack. Record the project URL and the anon / publishable key; you will enter them in both the browser-facing and the server-facing configuration.

3.2 Enable PostGIS before the first migration

> ⚠ **Ordering trap — enable PostGIS first**
>
> The first migration creates a summary view **only if PostGIS already exists**. If it is absent at that moment the view is silently skipped, and PostGIS is only installed later by migration `0012`.
>
> **Do this first:** Supabase → **Database** → **Extensions** → `postgis` → **Enable**. Then apply the migrations.

3.3 Apply the migrations

This project is not wired to a migration runner: there is no Supabase configuration file and no seed script, so `supabase db push` will not work. The migrations are applied as a hand-run, ordered sequence — 32 of the 33 files.

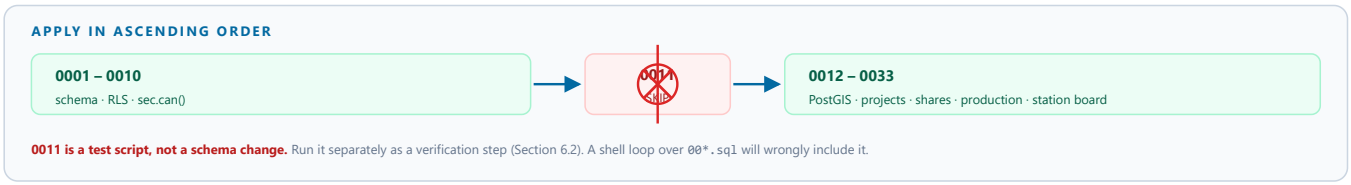


Figure 4. Migration order. Thirty-two files apply; `0011` is excluded.

Files to apply

ORDER	FILE	CREATES
1	`0001_schema_migrations.sql`	`subgrids`, `audit_logs`, `notifications`, `qa_defects`, `qaqc_audit_runs`, `project_settings`
2	`0002_foundation_production_migration.sql`	`datasets`, `processing_jobs`
3	`0003_foundation_processing_migration.sql`	Handoff and quality columns on `processing_jobs`
4	`0004_rls_application_tables.sql`	`user_accounts`, `deletion_requests`, `recycle_bin`
5	`0005_realtime_qaqc.sql`	Realtime quality publication, `security_invoker` views
6	`0006_file_inventory.sql`	`file_inventory`
7	`0007_hardening.sql`	Constraints, indexes, `survey_recycle_bin`
8	`0008_fix_security_advisor.sql`	Security Advisor remediations
9	`0009_security_functions.sql`	**The `sec` schema** — `can`, `get_app_role`, `is_role`, `normalize_role`

	0011_security_tests.sql	Not a migration — verification only (Section 6.2)
11	`0012_core_tables_and_rls.sql`	PostGIS, `panoramas`, `staging_panoramas`, `panoramas_view`
12	`0013_prune_blobloated_metadata.sql`	Strips oversized road-analysis state from user metadata
13	`0014_road_analysis_state_rpc.sql`	`sec.save_road_analysis_state()`
14	`0015_projects.sql`	`projects` registry
15	`0016_per_project_isolation.sql`	`project_id` across 12 tables, `projects_adopt_legacy_rows`
16	`0017_project_delete_cascade.sql`	`projects_delete_cascade()`
17	`0018_map_shares.sql`	`map_shares`, `map_share_touch()`
18	`0019_map_shares_public_access.sql`	Public read policy for shares
19	`0020_datasets_operator_delete.sql`	Operator delete rights on `datasets`
20	`0021_road_analysis_geometry_storage.sql`	**Creates the private `road-analysis-geometry` bucket**
21	`0022_production_runs.sql`	`production_runs`, attempts, releases, release files
22	`0023_station_board_items.sql`	`station_board_items`
23	`0024_stage_event_ledger.sql`	`stage_event_ledger`
24	`0025_station_board_metric_unit.sql`	Capture-point units on the station board
25	`0026_hub_session_state.sql`	`hub_session_state`
26	`0027_reconcile_panorama_columns.sql`	Latitude, longitude and heading columns, with backfill
27	`0028_sync_file_inventory.sql`	Storage-to-inventory triggers and backfill
28	`0029_csv_file_name.sql`	CSV filename column and backfill
29	`0030_role_precedence.sql`	**Directory row becomes authoritative for role and status** — makes **Disable** revoke access. Back-fills first, so it cannot lock anyone out
30	`0031_rls_completeness_backstop.sql`	Closes the two RLS gaps a fresh install leaves: `survey_recycle_bin` never had RLS enabled, and `batch_logs` had RLS but **no policies** (deny-all)
31	`0032_qa_defects_run_scope.sql`	**Required.** Adds `qa_defects.run_id` and widens the unique key to `(project_id, subgrid, run_id, point_id)`, so one run's defects stop appearing on a sibling run of the same subgrid
32	`0033_qa_defects_item_key.sql`	**Required.** Adds `qa_defects.item_key` where missing, back-fills it from `point_id`, then enforces `NOT NULL`
33	`0034_survey_metadata_filenames.sql`	**Required.** Creates `survey_metadata_filenames` table, tracking captured metadata files per survey run

34	`0034_repair_privileges_and_rls.sql`	**Required.** Repairs table privileges for `station_board_items`, `hub_session_state`, and `survey_metadata_filenames` to `authenticated`, and aligns capability gating
35	`0036_organizations_and_tenancy.sql`	**Required for multi-tenancy.** Creates `organizations` and `organization_members`, adds `user_accounts.is_platform_admin`, and defines `sec.current_org()` — the function every later tenant policy resolves against
36	`0037_tenant_attribution.sql`	**Required.** Adds `organization_id` attribution to the operational tables
37	`0038_tenant_rls_operational.sql`	**Required.** Tenant-scoped Row-Level Security on the operational tables via `sec.in_org()`
38	`0039_tenant_rls_production.sql`	**Required.** Tenant-scoped RLS on the production tables
39	`0040_tenant_rls_ledgers_and_rpcs.sql`	**Required.** Tenant-scoped RLS on the ledger tables; re-gates the project RPCs
40	`0041_tenant_storage_policies.sql`	**Required.** Tenant-scoped storage policies, object keys namespaced `orgs/{organization_id}/`
41	`0042_file_inventory_attribution.sql`	**Required.** Attributes `file_inventory` rows to an organization and repairs the storage trigger
42	`0043_organization_workers.sql`	**Required for multi-tenancy.** Per-company NAS worker endpoint and bearer token in `organization_workers`. Without it two companies sharing a database could browse each other's raw survey imagery
43	`0044_console_access_and_memberships.sql`	**Required for the Seller Console.** `console_access()`, `console_organization_members()`, `console_invite_member()`, `activate_pending_memberships()`. **Without it `/console` refuses to open**

0032 through 0044 are required, not optional

Stopping early produces a build that starts, reads, and quietly fails.

Without `0032` **&** `0033` the `qa_defects` select fails or defect upserts reject on missing columns.

Without `0035` table privileges on `station_board_items`, `hub_session_state`, and `survey_metadata_filenames` belong only to `postgres`. Application users and operators encounter silent permission denied errors.

Without `0036` **–** `0042` the tenant boundary does not exist: every company sees every other company's data.

Without `0044` the Seller Console cannot open. It reports `Could not find the function public.console_access in the schema cache` and refuses — a safe failure, but still a lockout.

All migrations are idempotent and safe to re-run. Apply the complete set.

File name check

The file at position 18 is `0019_map_shares_public_access.sql` — it grants **public** read access to share rows. Older notes refer to `0019_map_shares_auth_only.sql`, which does not exist.

CONSTRAINT	REASON
`0009` before `0010`, `0012`, `0014`	Later files call `sec.can()`, which `0009` defines
`0015` before `0016`	`0016` adds `project_id` columns and a helper function
`0018` before `0019`	`0019` re-grants and re-creates the policy
`0023` before `0025`	`0025` warns and exits if the station board table is absent
`0007` and `0016` before `0031`	`0031` backstops `survey_recycle_bin` and `batch_logs`, which those two create
`0034` before `0035`	`0034` creates `survey_metadata_filenames`; `0035` grants privileges
`0036` before `0037` – `0042`	Those migrations add `organization_id` columns and call `sec.current_org()`, which `0036` creates
`0036` before `0043`, `0044`	Both reference `organizations` and the `sec.*` guards it defines
Resolved: `0004` and `0012`	`0004` guards `panoramas` and `staging_panoramas` with existence checks so fresh installs run cleanly without halting; `0012` creates the tables and sets authoritative role-guarded RLS

How to run them

Fastest path: run the single consolidated bundle `supabase/bootstrap.sql` (generated via `npm run migrations:bootstrap`). Paste and run in the Supabase SQL editor in one step.

psql: execute the consolidated migration bundle:

```bash
# bash
export PGHOST=<db-host> PGDATABASE=postgres PGUSER=postgres PGPASSWORD=<password>
psql -v ON_ERROR_STOP=1 "$DATABASE_URL" -f supabase/bootstrap.sql
```

The migrations are written to be idempotent, so re-running a file that partially applied is safe. After the full sequence, run the verification in **3.4**.

```
-- security functions present (expect: can, get_app_role, is_role, normalize_role)
select proname from pg_proc p join pg_namespace n on n.oid = p.pronamespace
 where n.nspname = 'sec' order by 1;

-- PostGIS enabled
select extname from pg_extension where extname = 'postgis';

-- row-level security actually enabled on the privileged tables (expect several rows)
select relname, relrowsecurity from pg_class
 where relnamespace = 'public'::regnamespace and relrowsecurity
 order by 1;

-- the views the application reads
  select table_name from information_schema.views
   where table_schema = 'public'
     and table_name in ('panoramas_view', 'panoramas_subgrid_summary');

  -- GATE: directory-authoritative role resolver in place (expect NEW)
  -- OLD means 0030 did not apply, and Disable does not revoke access
  select case when prosrc like '%not Active%'
           then 'NEW' else 'OLD' end as resolver
    from pg_proc where proname = 'get_app_role';

  -- RLS gaps: expect ZERO rows in both (spatial_ref_sys is PostGIS's own)
  select c.relname as rls_missing
    from pg_class c join pg_namespace n on n.oid = c.relnamespace
   where n.nspname = 'public' and c.relkind = 'r'
     and not c.relrowsecurity and c.relname <> 'spatial_ref_sys';

  -- RLS on but ZERO policies = deny-all. Expect ZERO rows.
  select c.relname as no_policies
    from pg_class c join pg_namespace n on n.oid = c.relnamespace
   where n.nspname = 'public' and c.relkind = 'r' and c.relrowsecurity
     and not exists (select 1 from pg_policies p
       where p.schemaname = 'public' and p.tablename = c.relname);
```

If `panoramas_subgrid_summary` is missing, PostGIS was not installed when migration `0001` ran. Enable PostGIS and re-run the view block from `0001` (Section 3.2).

If the resolver reads **OLD**, the sequence stopped before `0030` — most often because `psql` was run with `ON_ERROR_STOP=1` and stopped at `0004`. Re-run the sequence tolerantly, then `0030`. If either RLS query returns rows, apply `0031`.

Which buckets to create, and the privacy decision you must make consciously.

4.1 Panorama bucket — create it manually, and make it public

No migration creates the panorama bucket. Create it yourself: Supabase → **Storage** → **New bucket**.

SETTING	VALUE
Name	`MMS_PIC`
Public bucket	**ON**

Storage Modes: Public vs Enterprise Private

Public Bucket (Default): Builds direct public CDN/storage URLs for zero API signing latency during public survey viewing.

Enterprise Private Bucket: Fully supported via `isPrivateBucket` in Admin Settings. The platform pre-signs and caches signed tokens in memory with a 50-minute safe validity window, resolving images through authenticated storage endpoints without viewer interruption.

Privacy decision — make it consciously

With a public bucket, the object path is the only protection on an image, and survey filenames typically encode the subgrid and frame index. If your imagery is confidential, agree an alternative with your integrator **before go-live**: either keep raw imagery off this bucket and serve previews through the NAS worker, or request the signed-URL capability as a development item.

4.2 Road-analysis geometry bucket — automatic

Migration `0021` creates a bucket named `road-analysis-geometry` with private access and authenticated-only storage policies. **No action is required.** If it is missing, Road Analysis geometry saves fall back to browser-local storage, so nothing breaks visibly — check for it if layer saves appear not to persist.

4.3 A bucket you do not need

Some older documentation instructs you to create a `vector_layers` bucket. The current code never reads or writes it. Skip it.

4.4 Bucket summary

BUCKET	HOW IT IS CREATED	PRIVACY
`MMS_PIC`	Manually, in the dashboard	**Public**
`road-analysis-geometry`	Migration `0021`	Private

Bootstrapping the first account, which cannot be done from inside the application.

5.1 Automated Provisioning & Manual Fallback

> ✓ **Turnkey Provisioning CLI (Recommended)**
>
> Provision the initial administrator in a single step using the automated script: `npm run admin:seed` (or provide `--email` and `--password` arguments). Then verify full system and database readiness with `npm run doctor`.

If you prefer manual configuration in the Supabase Dashboard, follow the four steps below.

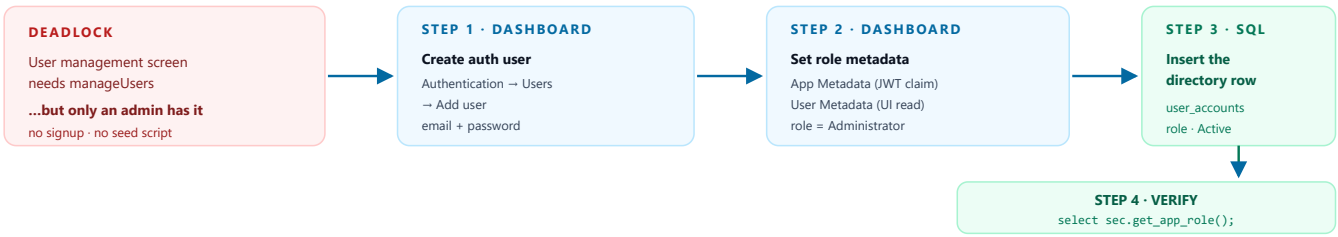


Figure 5. First-administrator bootstrap. All four steps are required, and step 3 must complete **before** the account's first sign-in.

5.2 The four steps

1 Create the authentication user

In the Supabase dashboard go to **Authentication** → **Users** → **Add user**. Enter an email address and password, or send an invitation. Note the address — it is used in step 3.

2 Set the role metadata

Open the new user and edit their metadata. Set **App Metadata** first, because that is what lands in the JSON Web Token and is the fastest path to a working administrator.

```
{ "role": "Administrator" }
```

Set **User Metadata** to the same value, because the dashboard reads User Metadata before App Metadata.

3 Insert the directory row

```
insert into public.user_accounts (name, email, role, status)
values ('<Full Name>', '<email>', 'Administrator', 'Active')
on conflict (email) do update
  set role = 'Administrator', status = 'Active';
```

4 Verify before signing in

```
select sec.get_app_role();   -- must return: Administrator
```

If it returns `Viewer`, one of steps 2 or 3 is missing or incomplete.

defaulting to Viewer. Running step 3 afterwards corrects it, but setting only the metadata afterwards does not, because the cached Viewer row already exists.

5.3 How the role is resolved, and why it can silently be Viewer

Two independent systems must agree, and neither reads the other:

SYSTEM	READS FROM	CONSEQUENCE
Dashboard interface	Authentication metadata on the signed-in user	Decides which controls are shown
Database	The token's role claim, then the `user_accounts` row	Decides which rows are visible

The dashboard tries, in order: `user_metadata.role`, `raw_user_meta_data.role`, `user.role`, `app_metadata.role`, `raw_app_meta_data.role`. For a real account, `user.role` holds the string `"authenticated"`, which is not an application role and normalises to **Viewer**. If no metadata key is set, the result is Viewer with no warning shown.

5.4 Adding the remaining users

Authentication lives entirely in Supabase. **Administration → Users does not create** accounts — it manages the role and access of people who already exist in Supabase.

1. **Create the account in Supabase**

   In the Supabase dashboard go to **Authentication → Users → Add user**, with their email address, a password, and **Auto Confirm User** enabled.

2. **Set the role as App Metadata**

   On that user, set **App Metadata** to the role they need:

   ```
   { "role": "Survey Operator" }
   ```

   Valid values are `Administrator`, `Survey Operator`, `QA Inspector` and `Viewer`. Optionally set **User Metadata** to the same value, because the interface reads User Metadata first.

3. **They appear on first sign-in**

   The user shows up in **Administration → Users** automatically the first time they sign in — that is when their directory row is written. Before then there is no row for them. From that point the dashboard manages their role and access.

Two things that catch people out

- **Set App Metadata, not only User Metadata.** The database resolves the role from the token's role claim, which Supabase sources from App Metadata. User Metadata alone makes the interface look correct while the database still sees `Viewer` — the user sees controls they cannot actually use.
- **Disable revokes permissions, not the account.** It sets the directory row to `Disabled`, which migration `0030` resolves to read-only. The Supabase sign-in account still exists. To stop someone signing in at all, remove or disable the user in Supabase.

The four roles, and — importantly — which controls the database actually enforces.

6.1 The four roles

ROLE	INTENDED FOR
Administrator	System owners. Full access, including user management and system settings.
Survey Operator	Field and production staff. Intake, pipeline operations, dataset management.
QA Inspector	Quality staff. Quality inspection and sign-off.
Viewer	Stakeholders. Read-only, plus report export.

6.2 Verifying the security boundary

Tier 1 — always runnable

Run the Section 3.4 queries, then paste and run all of `supabase/migrations/0011_security_tests.sql`. Its first, second and final sections must complete **without raising an exception**.

> ⚠ **The SQL editor cannot prove the boundary**
>
> The Supabase SQL editor runs as the `postgres` superuser, which **bypasses row-level security entirely**. Tier 1 proves the migrations applied and that role resolution is wired. It cannot prove that a Viewer is denied.

Tier 2 — a real boundary test

Requires a Viewer test account and a direct database connection:

```
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 <<'SQL'
BEGIN;
  SET LOCAL ROLE authenticated;
  SET LOCAL request.jwt.claims = '{"role":"authenticated","email":"viewer@test.local"}';
  select sec.get_app_role();   -- must print: Viewer
  insert into public.project_settings (id, settings)
    values ('rls_probe', '{}'::jsonb);
  -- EXPECT: ERROR  new row violates row-level security policy     => PASS
ROLLBACK;
SQL
```

The insert **must fail** with a row-level-security error. If it succeeds, the boundary is broken — stop and resolve before go-live. Then repeat with an Administrator token and the same statement: that one **must succeed**.

6.3 What is actually enforced

> 🔒 **Eight of twenty-one capabilities are enforced by the database**
>
> The remaining controls gate the **interface only** — the button is hidden or disabled, but a direct API call is not blocked. If your compliance requirements depend on server-side enforcement, treat the other thirteen as advisory and restrict roles accordingly.

ENFORCED BY ROW-LEVEL SECURITY	ADMINISTRATOR	SURVEY OPERATOR	QA / VIEWER

`runQaqc`	Yes	Yes	Yes
`deleteData`	Yes	Yes	No
`manageUsers`	Yes	No	No
`manageSettings`	Yes	No	No
`manageDatasets`	Yes	Interface only	Interface only
`reviewQaqc`	Yes	Interface only	Interface only
`approveDeletions`	Yes	No	No

Deletion approval is an interface guard

The `deleteData` capability is granted at the database level to **both** Administrator and Survey Operator. The "require administrator approval for delete" setting only gates the user interface. To enforce approval, do not grant the Survey Operator role where your process requires it.

Exposing the on-premises services to the dashboard without opening any port to the internet.

The dashboard is served over HTTPS from Cloudflare's edge. Your on-premises services sit on a private network. **Cloudflare Tunnel** bridges the two: `cloudflared` runs on each machine and makes **outbound-only** connections to Cloudflare.

> **No inbound firewall rule needed for the tunnel**
> Because `cloudflared` dials outward, nothing needs to accept an inbound connection. You may still restrict port 8000 at the firewall so that the services remain reachable only from the local network (Section 11.5).

7.1 Install cloudflared

```
# Windows
winget install --id Cloudflare.cloudflared

# Linux / macOS - see the Cloudflare downloads page for the current package
# https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/

cloudflared --version
```

7.2 Create one tunnel per machine

Create a tunnel **on each machine that hosts a service**: one on the worker PC, and one on each of the four station PCs. Five in total.

```
# on the worker PC
cloudflared tunnel login
cloudflared tunnel create nas-worker

# on each station PC, using a distinct name
cloudflared tunnel create pc1-agent
cloudflared tunnel create pc2-agent
cloudflared tunnel create pc3-agent
cloudflared tunnel create pc4-agent
```

`login` opens a browser to authorise your Cloudflare account. `create` prints a **tunnel identifier** and writes a credentials file under `~/.cloudflared/<TUNNEL-ID>.json`. **Keep that file** — it is the tunnel's identity, and the next step references it.

7.3 Write the ingress configuration

Each tunnel needs a configuration file mapping public hostnames to local services. Create `%USERPROFILE%\.cloudflared\config.yml` on Windows, or `~/.cloudflared/config.yml` elsewhere.

Worker PC

```
tunnel: <TUNNEL-ID>
credentials-file: C:\Users\<USER>\.cloudflared\<TUNNEL-ID>.json

ingress:
  - hostname: nas-api.example.com
    service: http://localhost:8000
```

```
tunnel: <TUNNEL-ID>
credentials-file: C:\Users\<USER>\.cloudflared\<TUNNEL-ID>.json

ingress:
  - hostname: pc1-agent.example.com
    service: http://localhost:8000
  - service: http_status:404
```

> **The trailing catch-all rule is required**
>
> `cloudflared` refuses to start unless the final ingress rule is a catch-all. Keep the `- service: http_status:404` line exactly as shown.

If you also want an embedded remote desktop for a station, add a second hostname pointing at your remote-desktop service (commonly `http://localhost:5901`) — see Section 8.7.

7.4 Point DNS at the tunnel

```
cloudflared tunnel route dns nas-worker  nas-api.example.com
cloudflared tunnel route dns pc1-agent  pc1-agent.example.com
cloudflared tunnel route dns pc2-agent  pc2-agent.example.com
cloudflared tunnel route dns pc3-agent  pc3-agent.example.com
cloudflared tunnel route dns pc4-agent  pc4-agent.example.com
```

This creates the DNS records automatically.

7.5 Run it as a service

Both options start the tunnel with the machine.

```
# Windows — registers a service that starts at boot
cloudflared service install

# then set the service to start automatically in services.msc
```

```
# Linux — systemd unit at /etc/systemd/system/cloudflared.service
[Unit]
Description=cloudflared
After=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/cloudflared --no-autoupdate run --token <TUNNEL-TOKEN>
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now cloudflared
```

The single-line **token** form is the simplest: in the Cloudflare Zero Trust dashboard, open the tunnel, copy the install command, and run it.

```
curl https://nas-api.example.com/health
```

An **HTTP 401 is the correct answer** when no token is supplied — it proves the tunnel is routing and the worker is enforcing its guard:

```
{ "detail": "Unauthorized." }
```

Now with the token, which should return the live response:

```
curl -H "Authorization: Bearer <NAS_WORKER_TOKEN>" https://nas-api.example.com/health

{ "status": "ok", "nas_base": "X:\\nas\\360_images", "worker": "NAS-PC-01" }
```

7.7 Optional — Cloudflare Access

For an additional gate at the origin, create a Cloudflare Access service token and set `CF_ACCESS_CLIENT_ID` and `CF_ACCESS_CLIENT_SECRET` (Section 8.4). The Pages Functions forward them automatically. This is defence in depth *on top of* the session check, not a replacement for it.

Build settings, the hosting constraint, and every variable to set.

8.1 Hosting constraint — read this first

Static-only hosting does not work with this platform

The private API — NAS scans, image previews, station probes, rename and upload — is implemented using the **Cloudflare Pages Functions** runtime: each handler exports an `onRequest` function and reads secrets from `context.env`.

Serving the built folder from nginx, Caddy or plain object storage **does not execute those handlers**. Under a static-only host:

- every private API route returns the application's HTML instead of JSON,
- the interface degrades **silently**, reporting "unavailable" rather than raising an error,
- and the session check that guards those routes does not run at all.

Deploy to Cloudflare Pages. A Pages-compatible edge runtime also works. If you must self-host, you need to build and maintain your own Pages-compatible server; that component does not ship with this product.

8.2 Create the project

Connect the repository in the Cloudflare dashboard:

SETTING	VALUE
Project name	Your choice
Root directory	**Repository root**
Build command	`npm run build`
Build output directory	`dist`
Production branch	`main` (or your deploy branch)

Root directory must be the repository root

Cloudflare detects the `functions/` directory there and deploys it automatically. If the root is set to `dist`, all seven function handlers are **silently omitted** and you get a dashboard with no private API and no error message. This is the single most damaging misconfiguration in this section.

A redirect rule in the project's public assets handles single-page route fallback for you. No server rewrite configuration is needed.

8.3 Build-time variables

These are **compiled into the browser bundle**. Set them under **Settings → Environment variables**. Only three are required.

VARIABLE	REQUIRED	PURPOSE
`VITE_SUPABASE_URL`	Yes	Project base URL. No fallback; the application warns if absent

`VITE_MAP_URL`	Yes	WebGIS application URL. **No code fallback** — unset means every embedded map is a blank frame
`VITE_WORKER_API_MODE`	No	`proxy` (default) or `direct`
`VITE_STATION_AGENT_MODE`	No	`proxy` in production, `direct` in development
`VITE_SUPABASE_BUCKET`	No	Defaults to `MMS_PIC`
`VITE_SENTRY_DSN`	No	Error reporting. Omit to disable entirely

Never put a secret in a build-time variable

Anything prefixed `VITE_` is inlined into the shipped bundle and is readable by anyone who views the page source. A service-role key, the NAS worker token or a station-agent token placed here would be **public**. Server secrets belong in Section 8.4.

Changing a build-time variable requires a rebuild

If you add a variable **after** a deployment, the running site is unchanged. Trigger a new build — push a commit, or use retry in the Pages dashboard.

A retired name that does nothing

`VITE_NAS_API_ENABLED` appears in some older notes but is **not read by any code**. Setting it has no effect. Proxy behaviour is derived from `VITE_WORKER_API_MODE` and the presence of the runtime secrets in Section 8.4.

8.4 Runtime variables and secrets

Read **server-side** by the Pages Functions; never sent to the browser. Mark them **Secret**.

VARIABLE	REQUIRED	PURPOSE
`SUPABASE_URL`	Yes	Session validation target
`SUPABASE_ANON_KEY`	Yes	Session validation key
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Server-side only; never sent to the browser. Required by the Seller Console's invite function (it creates the sign-in account) and by per-company NAS worker resolution. **Without it, `/api/console-invite` returns 503 and no invitation can be created.**
`NAS_API_URL`	Yes	Worker tunnel origin, **no trailing slash**
`NAS_WORKER_TOKEN`	Yes	Must match the worker's token exactly
`NAS_IMAGE_TOKEN_SECRET`	No	Signing key for short-lived preview URLs; falls back to the worker token
`STATION_AGENT_URLS`	For the board	JSON map of station id to HTTPS agent origin
`STATION_AGENT_TOKENS`	No	JSON map of station id to that agent's token
`STATION_AGENT_TOKEN`	No	Single fallback token
`CF_ACCESS_CLIENT_ID`	No	Cloudflare Access service token, forwarded upstream

```
{ "blur":      "https://pc1-agent.example.com",
  "stitch":    "https://pc2-agent.example.com",
  "lightroom": "https://pc3-agent.example.com",
  "photoshop": "https://pc4-agent.example.com" }
```

8.5 Deploy

Push to the connected branch. Cloudflare Pages builds automatically — there is **no deploy automation in this repository**, so the Git integration is the entire mechanism.

8.6 Local development

```
npm install
cp .env.example .env          # Windows PowerShell: copy .env.example .env
```

Set these so the development server proxies the on-premises API exactly as production does:

```
VITE_SUPABASE_URL=https://<project>.supabase.co
VITE_SUPABASE_ANON_KEY=<anon key>
VITE_MAP_URL=<your WebGIS URL>

NAS_API_URL=https://nas-api.example.com
NAS_WORKER_TOKEN=<same token as the worker>
```

The last two are deliberately **not** prefixed with `VITE_`. The development server reads them and injects the authorisation header, so the browser never receives the token. If both are absent, no proxy is registered and the interface reports "worker proxy is not configured" rather than silently guessing a NAS address.

```
npm run dev
```

8.7 Remote desktop (optional)

To embed a station's live desktop, run a browser-based remote desktop service on that PC, expose it through a tunnel hostname (for example `pc2-remote.example.com`), and set the station's remote desktop URL in the application's provider settings.

A private `http://192.168.x.x` address will not work — the dashboard is served over HTTPS and browsers block mixed content. Native remote-desktop file downloads for operators on the local network continue to work.

8.8 Request path

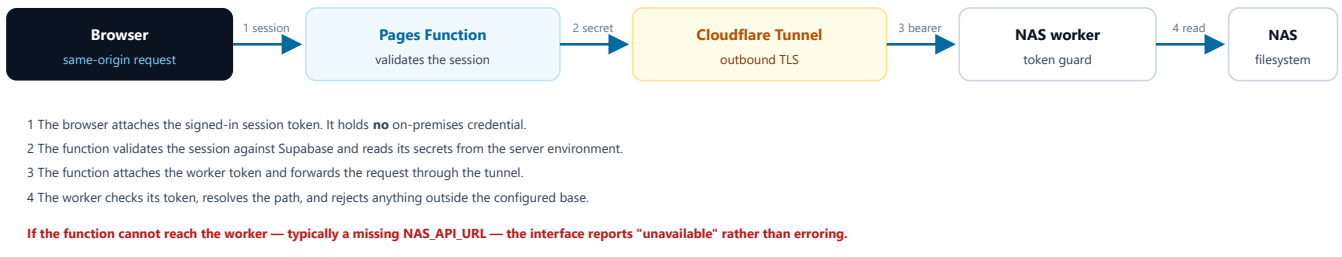


Figure 6. How a private request reaches the NAS, and where each credential is applied.

The folder structure the services read. Getting this wrong fails quietly, not loudly.

The worker and the agents read the survey filesystem directly. By default, the system follows a standard production stage layout, with built-in **adaptive dynamic stage discovery** supporting alternative grid folders (e.g. `Grid 2`, `Grid 3`), flat stage layouts, custom stage aliases, and environment variable overrides (`NAS_STITCH_STAGE`, etc.). The recommended production layout is shown below.



Figure 7. Required folder tree. Colours mark the depth rules: amber `Grid 1` (literal, with a space), violet `Project-OUT`.

9.1 The depth rules

Three different conventions are in play. Mixing them up fails silently.

STAGE FOLDER	NEEDS PROJECT-OUT?	NEEDS GRID 1?	USED BY
`00_Raw_data`	No	Yes	Worker + agent
`01_Metadata`	No	Yes	Worker
`02_Blurring`	No	No	Agent only
`03_Stitching`	Yes	Yes	Worker + agent
`04_Lightroom`	No	No	Agent only
`05_Final`	Yes	Yes	Worker
`DELIVERABLES`	No	No	Agent only

Three things that commonly go wrong

- `Grid 1` **contains a space.** On Linux, confirm the mount preserves it.
- **The run folder name is a join key.** It links stitched imagery to its telemetry, so the run folder under `03_Stitching` and the run folder under `01_Metadata` must be **identical**. If they differ, coordinates will not be paired to imagery.
- `02_`, `04_` **and** `DELIVERABLES` **have no depth rule.** Their first-level directories are read directly as subgrids.

Use the **uppercase GIS form**, for example `N93E070` or `N01E037`. The platform applies three different subgrid patterns for three different purposes, and the uppercase GIS form is the only shape that satisfies all three. Lowercase works on the worker but is handled inconsistently by the agents.

9.3 Access rights

ACCESS	REQUIREMENT
Read the whole survey tree	**Required — worker and all four agents**
Write to `DELIVERABLES`	**Required — bucket upload**
Write to `panoramas/`	Only for the rename action

The read-only service that exposes the survey filesystem to the dashboard.

10.1 Install

```
# Windows PowerShell
cd <repository>\worker
py -3.10 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
# Linux
cd <repository>/worker
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Dependencies are deliberately minimal — a web framework, a server and a configuration loader. There is no image or scientific processing stack to install, and no model weights to download.

10.2 Configure

```
copy .env.example .env          # Windows PowerShell
```

```
# Root of the survey tree. Every path the dashboard sends resolves under this,
# and any path escaping it is rejected with HTTP 400.
NAS_BASE_PATH=X:\nas\360_images

# Shared secret. Required on any non-loopback bind. The Pages Functions and
# the development proxy inject this header server-side; the browser never sees it.
NAS_WORKER_TOKEN=<generate a long random string>

# Seconds a storage response is cached before the volume is re-probed.
STORAGE_CACHE_TTL=30

# 1 = structured JSON logs to stdout.
NAS_LOG_JSON=0

# Identifier reported on /health, so several workers behind one endpoint
# can be told apart. Defaults to the machine hostname.
NAS_WORKER_ID=
```

Generate a token with:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

> ⚠ **If the token is blank, the worker accepts unauthenticated requests**
>
> Only safe on a loopback bind with the tunnel in front. Never leave `NAS_WORKER_TOKEN` empty on a machine reachable from the local network.

```
uvicorn app:app --host 127.0.0.1 --port 8000
```

Bind to **loopback only**. The Cloudflare Tunnel on the same machine terminates TLS and is the sole public entry point. Do not bind `0.0.0.0` unless you also restrict the port at the firewall.

Run it as a service for autostart, using the same pattern as the agents in Section 11.6.

10.4 Verify

```
curl http://127.0.0.1:8000/health
```

```
{ "status": "ok", "nas_base": "X:\\nas\\360_images", "worker": "NAS-PC-01" }
```

Then test an authenticated data route:

```
curl -H "Authorization: Bearer <NAS_WORKER_TOKEN>" \
  "http://127.0.0.1:8000/api/nas-scan?action=subgrids"
```

Expected — your subgrid appearing here confirms the folder tree is correct:

```
{
  "success": true,
  "basePath": "X:\\nas\\360_images",
  "detectedSubgrids": ["N93E70"],
  "subgrids": [{ "code": "N93E70", "existsInStitching": true, "label": "N93E70" }]
}
```

An empty subgrid list means the tree does not match

Walk up from `NAS_BASE_PATH` and confirm that `03_Stitching/Project-OUT/Grid 1/` exists, spelled exactly as shown — this is the most common cause.

Also check storage telemetry:

```
curl -H "Authorization: Bearer <NAS_WORKER_TOKEN>" http://127.0.0.1:8000/api/storage
```

10.5 Run the worker tests

```
cd worker
PYTHONPATH=. python -m pytest tests -q
```

This covers the conditional-request behaviour of image previews and the six scan actions. The configuration loader must be installed, or the HTTP tests skip silently.

One small service per workstation, reporting progress and performing local actions.

11.1 Station roles

IDENTIFIER	PC	SOFTWARE	OUTPUT STAGE (DEFAULT)	READS
`blur`	1	Privacy / blur tool	`02_Blurring`	`00_Raw_data`
`stitch`	2	Stitching software	`03_Stitching`	`02_Blurring`
`lightroom`	3	Lightroom Classic	`04_Lightroom`	`03_Stitching`
`photoshop`	4	Photoshop batch	`05_Final`	`04_Lightroom`

Only these four values are accepted. Any other value produces an empty stage and a watch error.

11.2 Install

On **each** of the four PCs:

```
copy <repository>\station-agent C:\station-agent /r /e

cd C:\station-agent
py -3.10 -m venv .venv
.\.venv\Scripts\python -m pip install -r requirements.txt
```

Dependencies are a web framework, a server, a configuration loader and **process-inspection**. The process library matters:

> ⚠ **Install the process library even though the service starts without it**
>
> Without it the agent cannot detect workstation processes, so the multi-station board never leaves "awaiting agent", and per-station CPU, memory and disk telemetry is absent. The service still runs, so the failure is silent.

> ⓘ **Optional GPU telemetry**
>
> To show GPU cards on the PC monitoring screen, install the GPU bindings as well — they are not in the requirements file:
>
> ```
> .\.venv\Scripts\python -m pip install pynvml
> ```
>
> This needs a vendor GPU driver. Without it the card falls back to a browser-reported approximation.

11.3 Configure

```
copy .env.example .env
```

```
# The NAS root AS MAPPED ON THIS PC (drive letter or UNC path).
WATCH_ROOT=X:\nas\360_images

# Overrides, only if your layout differs from Section 9.
# OUT_STAGE=03_Stitching
# IN_STAGE=02_Blurring

# 1 = count capture POINTS instead of stitched images.
# Defaults to 1 for blur, 0 for the others.
# POINT_MODE=0

# Substrings matched against running process names/paths, comma separated.
# Match your actual software.
PROCESS_NAMES=ptgui.exe,ptguipro.exe

# Watch loop period, and the "file is still growing" window in seconds.
SCAN_INTERVAL_SEC=5
GROWING_WINDOW_SEC=90

# Shared secret. Required unless the agent is unreachable from anywhere
# but this LAN.
AGENT_TOKEN=<generate a long random string per station>

# Bind address and port. Keep 0.0.0.0 only with a firewall rule.
AGENT_HOST=0.0.0.0
AGENT_PORT=8000
```

Use a **different token per station**, and register each one in `STATION_AGENT_TOKENS` (Section 8.4).

11.4 Verify

```
.\.venv\Scripts\python app.py
```

In a second shell:

```
curl http://localhost:8000/health
```

```
{
  "status": "ok", "agent_version": "1.0.0", "station_id": "stitch",
  "hostname": "PC2", "uptime_sec": 86400,
  "cpu_usage": 12, "cpu_cores": 16, "ram_total_gb": 32, "ram_used_gb": 14.2
}
```

Then check the station payload:

```
curl -H "Authorization: Bearer <AGENT_TOKEN>" http://localhost:8000/api/station
```

Confirm two things:

- the watch error is **null** — meaning `WATCH_ROOT` resolved on this PC, and
- the output section lists your subgrids.

If a watch error is present, the path is wrong for this machine. A drive letter that exists on one workstation frequently does not exist on another.

The agent binds port 8000 with no transport encryption of its own. Restrict it to the local network:

```
New-NetFirewallRule -DisplayName "GeoSphere Station Agent" `
  -Direction Inbound -LocalPort 8000 -Protocol TCP `
  -Action Allow -Profile Domain
```

The Cloudflare Tunnel is outbound-only and needs no inbound rule.

11.6 Autostart

Windows — Task Scheduler → Create Task:

- Trigger: **At startup**
- Action: **Start a program** → `C:\station-agent\run_agent.bat`
- Start in: `C:\station-agent`

Linux — a systemd unit pointing at the virtual environment's server binary.

Configuring the application itself, including one step that currently has no screen.

> **The platform is deployed once. Clients are onboarded through the console.**
>
> Everything above this point — accounts, the Supabase project, the migrations, the first administrator, the Pages deploy — is a **once-per-platform** task performed by the Seller. A client company is then onboarded in the **Seller Console** at `https://app.geosphere.my/console`, and never installs the platform themselves. Only their survey hardware (NAS, worker, four station agents) is theirs.

12.1 Onboarding a client company

12.1.1 Who may open the console

Only a **platform administrator**. The decision is made by the database (`public.console_access()`), not by the interface: the console refuses to render anything until that read answers. A Viewer or Survey Operator who navigates to `/console` sees a refusal screen, and if the check cannot reach the database at all the screen says the answer is **unknown** rather than guessing either way.

12.1.2 Create the company

Seller Console → Companies → Add Client Company. Give it a name, a unique client code, a contact email, a status, a licensed seat count and a storage quota, then choose the database routing.

A **suspended** or **expired** company grants nobody access, even to people whose invitations have already been accepted.

12.1.3 Invite the staff

Seller Console → Team Members → select the company → Invite Member. One address at a time, with a company role: `CompanyAdmin`, `Operator`, `Inspector` or `Viewer`.

The invitee appears immediately, in one of four states. They are deliberately not collapsed into a single "registered" badge, because the distinction is the whole point:

STATE	MEANING
Invited — awaiting confirmation	An account exists, but the person has not confirmed the address. **Grants no access.**
Confirmed — awaiting first sign-in	The address is verified. Access begins when they first sign in.
Active	Signed in and working in the company.
Suspended	Revoked. That company can no longer read their data.

An invitation grants nothing on its own. Access begins only after the person confirms their email address and signs in for the first time. This is deliberate: if confirmation is not required, anyone who can register an address can claim the company that owns it.

12.1.4 What to tell the client

They receive a confirmation email, click it, and sign in at `https://app.geosphere.my`. On first sign-in their membership activates automatically. If they sign in before confirming, they see an **"Awaiting workspace assignment"** screen explaining that their account is verified but not yet a member of any company — which is the correct instruction to give them: ask you.

to project team members, so a client address will often receive nothing. Configure real SMTP (Supabase → Authentication → Providers) and test once with a non-team address. If the mail cannot be delivered, the console says so ("User created, but the confirmation email could not be sent") rather than reporting a successful invitation.

> ⚠ **What the platform does not enforce**
>
> The tenant boundary **is** enforced by PostgreSQL and verified by an executable cross-tenant gate. These commercial controls are **reported but not enforced**:
>
> • **Named user seats** — shown beside confirmed members, with over-limit companies highlighted. No write is refused.
>
> • **Named storage quotas** — an allocation figure. Actual object usage is not measured, so an over-quota company is not detectable.
>
> • `dedicated_external` **routing** — credentials are stored and readable, but the application does not connect to that instance. Route new companies to `cloud_shared`.
>
> • **Tenant export and offboarding deletion** — not available.

12.2 Create your first project

A project is **mandatory** — every query in the platform is scoped to the active project, and the dashboard forces project selection immediately after login. The wizard's only required field is the campaign name.

12.3 Required settings

Open **Settings** → **Project and Map Settings**. The form is read-only unless you are an Administrator.

SETTING	WHY IT IS REQUIRED
Supabase REST endpoint URL	No database client without it
Public anon API key	No sign-in without it
Database host provider	Selects your provider topology
Storage provider	Selects the panorama URL builder
Provider bucket / domain / container	Without it, publishing stays disabled
Default basemap provider	Map backdrop

Storage settings vary by provider — a bucket name, a CDN domain, or a bucket and region pair.

> ⓘ **Build-time variables are not set here**
>
> The Supabase and map variables are **build-time** settings configured on Cloudflare Pages (Section 8.3). This screen only pre-fills the form; it does not set them. Changing them here has no effect on the running application.

12.4 Two settings in an unexpected place

The working directory and worker endpoint are **not** on the admin screen. Set them in **NAS and Daemon** → **Capacity and Volumes**:

SETTING	PURPOSE

Worker URL	Endpoint used when not proxying

12.5 Station address configuration

> ✓ **No database step required**
>
> Station addresses are set in the interface: **Settings → Project & Map Settings → 4. Production Pipeline & Workstations**. Enter the address for each of the four processing PCs and press **Save**.
>
> Earlier revisions had no screen for this and required a settings-table SQL update, tracked as limitation 14.1. That defect is fixed. If you are upgrading, no migration is needed — fill in the addresses and save.

FIELD	PURPOSE	DEFAULT
IP address or hostname	Where the dashboard polls the station agent	blank → station reports *Unconfigured*
Agent port	Station agent listen port	`8000`
Live desktop channel	`RDP (mstsc)` or `noVNC (browser)`	`RDP`
RDP port	Port for the one-click `mstsc` launch	`3389`
noVNC port	websockify port for the in-browser desktop pane	blank
noVNC public URL	HTTPS noVNC endpoint; accepts `{ip}` and `{port}`	blank
Agent token	Shared secret matching the PC's `AGENT_TOKEN`	blank
Enabled	Whether the station is part of the active pipeline	on

The section header shows **N of 4 stations configured**. A station with no address reports itself as **Unconfigured** and the rest of the board keeps working, so PCs can be brought online one at a time. A mistyped address is flagged *Check address* rather than blocked, so a hostname that does not resolve to a dotted IP will not stop you saving.

> ⓘ **The live desktop pane needs a public URL on HTTPS**
>
> A browser blocks a private `http://` VNC iframe as mixed content, so on an HTTPS deployment the pane requires the public noVNC URL above. Leave it blank and the pane stays hidden for that station; RDP launch still works from the office LAN.
>
> Also sign out and back in after saving. The dashboard caches project settings in browser storage, so a hard refresh or a fresh session is needed to pick up the change. The stored settings type now declares the address and port keys, so exporting and re-importing project settings no longer drops them — a defect that affected earlier builds.

Topology is not editable here. Station order, software labels and the NAS stage folder templates come from the shipped four-station topology so the pipeline contract cannot drift per site. Only the address and remote-access fields are per-site.

In **proxy** mode — the default — the agents are reached through the Pages Functions using the hostname map, so the addresses matter less. Set them anyway so that local network fallback and the monitoring views work.

12.6 Roles and capabilities

Grant capabilities per operator in **Administration → Roles**. Remember that only eight capabilities are enforced by the database (Section 6.3) — do not rely on the others for compliance.

Two acceptance tiers, and the sign-off record.

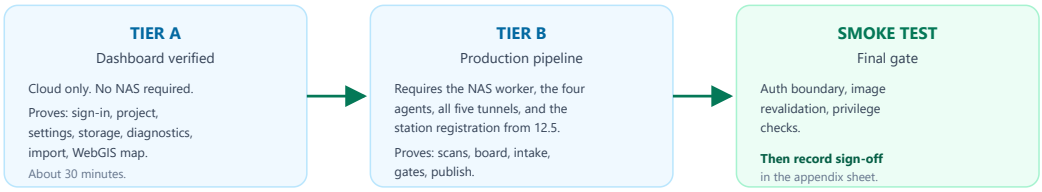


Figure 8. Verification tiers. Tier B depends on Tier A plus the on-premises half; the sign-off sheet is in the appendix.

13.1 Tier A — dashboard verified

About thirty minutes. Requires no NAS. Capture a screenshot at each pass signal for the acceptance record.

#	ACTION	PASS SIGNAL
1	Open the deployed URL	Landing page renders with the globe animation
2	Sign in as the seeded Administrator	No error banner
3	Project picker appears	Zero projects forces the wizard
4	Complete the wizard — campaign name is the only required field	Loading indicator, then the dashboard
5	Expand the left navigation rail	All workspaces listed; Settings and Administration unlocked
6	**Settings → Diagnostics → Ping**	**Connected**, database **operational**, storage **operational**, a real latency figure
7	Expand **Runtime Environment**	Database and map variables show values, not "not set"
8	**Administration → System Health → Run Diagnostics**	Database **operational** with a real millisecond figure
9	**Administration → Users**	Your row present with role **Administrator**
10	**Administration → Roles**	The capability matrix renders
11	Settings → **Test database connection and latency**	A number replaces "not measured"
12	Settings → set the provider bucket → **Verify manifest**	**Manifest OK** with a frame count
13	**Save** settings	Success confirmation, and an entry in the audit log
14	**Data Management** → import a two-row CSV with `filename`, `latitude`, `longitude`	Columns auto-map; duplicate check reports 0
15	**Road Analysis** → **Import Data** → add a small GeoJSON	Feature count renders; transport badge reads **worker**
16	**Dashboard**	The embedded WebGIS renders a basemap
17	Header → audit log icon	The step-13 entry appears

Expected cosmetic artefact — not a fault

The diagnostics panel reports **WebGIS: unknown** and **Realtime: unknown** even on a perfectly healthy installation, because those probes are not implemented. Ignore them and judge health from the database, storage and latency

Cloudflare Pages and rebuild (Section 8.3).

13.2 Tier B — production pipeline verified

Tier A cannot prove the on-premises half. Continue once the worker, agents and tunnels are running.

#	CHECK	PASS SIGNAL
1	Scan for subgrids through the tunnel	Your subgrid is listed
2	**NAS and Daemon** workspace	Folder browser lists the stitching stage; storage overview shows the volume
3	**Production Hub** → flight board	All four cards show an agent, not "awaiting agent"
4	**PC Monitoring**	Live processor, memory and disk per station
5	**Production Hub** → intake and pairing	Subgrid detected, survey folders listed, telemetry rows paired to imagery
6	Quality inspection station	Frames step through the 360 viewer; a defect flag writes to the ledger
7	Cloud bucket gate	Inventory verified and gate approved
8	WebGIS release gate → publish	Rows land in the published table
9	**Main Dashboard**	The published trajectory renders

⚠ **Steps 3 and 4 depend on Section 12.5**

Without the station registration those cards cannot reach the agents, however correctly the tunnels are configured.

⚠ **Bucket gate limitation**

Inventory verification works only with Supabase object storage. Other providers cannot complete that gate today — see item 14.3.

13.3 Final smoke test

Run the repository's smoke checklist, which adds the authentication boundary, image revalidation and privilege checks. Record the outcome in the appendix sheet.

Product defects that affect a client installation. Read before go-live.

14.1 Station addresses must be set before the board is useful FIXED

Impact. Until each station has an address saved, the flight board reports that station as "Unconfigured", the rename action is unavailable for it, and bucket upload for non-Supabase providers cannot run for it. A station without an address does not affect the rest of the board, so PCs can be brought online one at a time.

Resolution. Earlier revisions had no screen for this and required a settings-table SQL update, and a settings export silently dropped the address and port keys. Both are fixed: set the addresses in **Settings → Project & Map Settings → 4. Production Pipeline & Workstations** (Section 12.5), and the stored settings type now declares those keys. No migration is required when upgrading.

14.2 Bucket upload works for all cloud providers FIXED

Resolution. In earlier revisions, chained shell operators caused CLI invocations to fail on non-R2 providers. Station Agent (`station-agent/app.py`) now executes commands in structured sequential `(argv, desc)` loops with error trapping, supporting all 7 backends: Cloudflare R2, AWS S3, Wasabi, Google Cloud Storage, Azure Blob, Supabase CLI, and NAS local copy.

14.3 Bucket gate verification supports multi-cloud & pagination FIXED

Resolution. The Cloud Bucket Gate (`BucketPublicationGate.tsx`) now features an automated pagination loop (up to 1,000 objects per page) for Supabase Storage, and multi-cloud candidate detection for Cloudflare R2, AWS S3, and Wasabi deliverable files.

14.4 The in-app tour and help text are out of date

Impact. The first-run interactive tour and the help dialog still describe the pre-refactor interface. They will mislead operators and new staff.

Workaround. Dismiss the tour and train against this guide instead.

14.5 Session inactivity lockout in embedded 360 viewers FIXED

Resolution. In earlier revisions, user interactions inside WebGIS and PhotoSphereViewer iframes were invisible to parent window event listeners. An activity bridge in `src/App.tsx` now listens for `window.blur` and bidirectional `geosphere:activity` events, resetting the timeout during active inspection.

14.6 Deletion approval is an interface guard

Impact. The delete capability is granted at the database level to both Administrator and Survey Operator, so the approval queue can be bypassed with a direct API call.

Workaround. Do not grant the Survey Operator role where your process requires approval.

14.6b Disable revokes permissions, but not the sign-in account

Impact. Disable in Administration → Users sets the directory row to `Disabled` . Since migration `0030` that resolves to read-only at the database level, so the control genuinely revokes

the revocation, and role resolution would fall back to the token claim, restoring the elevated role.

Workaround. To stop someone signing in at all, remove or disable the user in **Supabase → Authentication → Users**.

14.7 Retired configuration names

NAME	STATUS
`VITE_NAS_API_ENABLED`	Never existed in code. Documented in older notes
`CONCURRENCY`, `MAX_ACTIVE_JOBS`, `MAX_QUEUE_DEPTH`, `WORKER_JOB_DB`	Retired with the job queue. Setting them does nothing
`MASK_NEIGHBOR_WARP`, `LAZY_MODEL_DOWNLOAD`	Retired. Not read anywhere
Worker job, release and metrics endpoints	Removed. A "not found" response is expected, not a fault
Worker port `8787`	Never correct. Use `8000`
`0019_map_shares_auth_only.sql`	No such file. The real file is `0019_map_shares_public_access.sql`

14.8 Renaming requires a panoramas folder

Impact. The NAS rename action targets the `panoramas/` subfolder of a survey run. If it does not exist the request is rejected.

Workaround. Use the tiled stitched layout (Section 9.1), or rename directly on the filesystem.

Symptom to cause. Entries marked silent fail without an error message.

> ⚠ **This platform prefers to report "unavailable" over raising an error**
>
> Several failures are therefore silent. When something is not working, check the table below before assuming a data problem.

SYMPTOM	LIKELY CAUSE	FIX
Blank frame where the map should be (silent)	Map variable unset	Set it on Pages, then **rebuild** (8.3)
"Worker proxy is not configured" (silent)	Proxy variables absent in the development file	Add both, restart the dev server (8.6)
Edge returns **503** on private routes	Runtime secrets missing	Set the API URL and worker token (8.4)
Edge returns **401** on a private route	Session missing or expired	Sign in again; confirm the server Supabase variables
Tunnel hostname returns **404**	Ingress rule does not match the requested hostname	Check the ingress rule; restart the tunnel service
Tunnel hostname returns **401** and nothing else works	**Correct** — the token is required	Add the authorisation header; if it works the tunnel is fine (7.6)
Tunnel hostname keeps changing	A quick tunnel was used	Create a **named** tunnel with DNS routing (7.2)
Station cards show **no-ip-configured**	Address not saved for that PC	Set it in Settings → 4. Production Pipeline & Workstations (Section 12.4)
Station cards show **unreachable**	Tunnel, firewall, or agent not running	Check the agent is up, the ingress matches, port 8000 is allowed
Station cards show **not-reporting**	Agent answers but its watch loop is failing	Check the watch root on that PC (11.4)
Agent exits immediately at startup	Station identifier missing or invalid	Must be one of the four values in 11.1
No processor or memory figures on a station	Process library not installed	Install the requirements on that PC (11.2)
No GPU card on the monitoring screen	GPU bindings not installed	Install the optional GPU package (11.2)
Panoramas render broken (silent)	Bucket is private, or the wrong bucket name	Make it public; re-check the setting (4.1)
"No resolvable panorama URL" in quality review	Storage provider or bucket incomplete	Re-check Section 12.3
Subgrid scan returns an empty list (silent)	Folder tree mismatch, or base path wrong	Re-check Sections 9.1 and 10.2
Coordinates not paired to imagery (silent)	Run folder names differ between the two stage folders	Make them identical (9.1)
Publish button permanently disabled	Provider bucket not configured	Set the bucket in Settings (12.3)
Cannot reach the Administration workspace	Seeded role resolved to Viewer	Verify the role query (5.2)

Migration error mentioning a security function	Applied out of order	Re-apply in order (3.5)
Every survey run reports **0 defects**	`0032` not applied, or PostgREST serves a stale schema	Apply `0032`. If the column exists, `NOTIFY pgrst, 'reload schema';` or restart PostgREST. The select error is caught, so nothing is logged — the count is absent, not zero
`Could not find the 'run_id' column`	Missing `0032`, or a stale PostgREST schema cache	Apply `0032`, then `NOTIFY pgrst, 'reload schema';`
`item_key … violates not-null constraint` on a QA/QC write	`0033` not applied	Apply `0033` — it adds the column and back-fills from `point_id`
Summary view missing after migrations	PostGIS was not enabled before the first migration	Enable PostGIS, re-run the view block (3.2)
Diagnostics show WebGIS or Realtime **unknown**	**Expected** — probe not implemented	Ignore (13.1)
Database changes not appearing after an edit	Settings cached in the browser	Sign out and back in, or hard refresh

Lookup tables. Precedence is project settings, then environment variables, then built-in defaults.

16.1 Browser build-time variables

VARIABLE	REQUIRED	PURPOSE
`VITE_SUPABASE_URL`	**Yes**	Project base URL
`VITE_SUPABASE_ANON_KEY`	**Yes**	Public anon / publishable key
`VITE_MAP_URL`	**Yes**	WebGIS application base. No fallback
`VITE_SUPABASE_KEY`	No	Legacy alias for the anon key
`VITE_WORKER_API_MODE`	No	`proxy` (default) or `direct`
`VITE_STATION_AGENT_MODE`	No	`proxy` in production, `direct` in development
`VITE_SUPABASE_BUCKET` / `VITE_STORAGE_BUCKET`	No	Panorama bucket. Default `MMS_PIC`
`VITE_STORAGE_PROVIDER`	No	Storage provider selection
`VITE_R2_BUCKET` / `VITE_R2_DOMAIN`	No	Cloudflare R2 target
`VITE_IMAGE_CDN_URL`	No	Generic image CDN base
`VITE_S3_BUCKET` / `VITE_S3_REGION`	No	S3 target. Region default `ap-southeast-1`
`VITE_GCS_BUCKET`	No	Google Cloud Storage target
`VITE_AZURE_ACCOUNT` / `VITE_AZURE_CONTAINER`	No	Azure target
`VITE_WASABI_BUCKET` / `VITE_WASABI_REGION`	No	Wasabi target. Region default `us-east-1`
`VITE_NAS_SERVER_URL`	No	Direct NAS image origin
`VITE_NAS_WORK_BASE_PATH`	No	Default NAS working directory
`VITE_PRODUCTION_API_URL`	No	Worker base URL — only when mode is `direct`
`VITE_PRODUCTION_API_KEY`	No	Static worker secret — direct mode only. Prefer proxying
`VITE_ROAD_GEOMETRY_BUCKET`	No	Private layer geometry bucket
`VITE_ROAD_EXTRACTION_ROUTE`	No	`overpass` (default) or `custom`
`VITE_ROAD_EXTRACTION_URL`	No	Road-network endpoint, browser fallback only
`VITE_ROAD_EXTRACTION_PROXY`	No	Server-side proxy — preferred
`VITE_ROAD_EXTRACTION_DIRECT`	No	Bypass the proxy for debugging
`VITE_MAPLIBRE_WORKER_URL`	No	Self-hosted map worker for strict content policies
`VITE_SHARE_LIVE_MODE`	No	Embed the live map in share pages

`VITE_DB_*` (eight variables)	No	Table and view name overrides
`VITE_SENTRY_DSN`	No	Error reporting. Omit to disable
`VITE_DATA_QUIET`	No	Suppress non-fatal analysis console noise
`VITE_BRAND_NAME`	No	Product wordmark in the logo lockup. Default `GeoSphere`
`VITE_BRAND_MARK`	No	Trailing accent mark. Default `360°`; set **empty** to omit
`VITE_BRAND_URL`	No	Canonical origin for canonical / OG / Twitter / JSON-LD URLs. No trailing slash
`VITE_BRAND_PROJECT_NAME`	No	Seeded project name on a fresh install
`VITE_BRAND_CONTRACT_CODE`	No	Seeded contract code on a fresh install
`VITE_BRAND_CLIENT_NAME`	No	Seeded client name on a fresh install

Rebranding a deployment

The six `VITE_BRAND_*` variables replace the product name, the browser tab title, every generated report and PDF footer, and the canonical / Open Graph / Twitter / JSON-LD metadata. Logo **artwork** is not covered — overwrite `public/branding/` at deploy time if you need your own.

All six default to the shipped values, so leaving them unset reproduces the default build exactly. Blank means "not configured" for five of them; `VITE_BRAND_MARK` is the exception, where an empty value is the instruction to omit the mark.

They are **build-time** — Vite inlines them into `dist/`, so changing them requires a rebuild, not a settings edit. Same constraint as `VITE_SUPABASE_URL`: **one build per deployment**. The three seeded values are only what an untouched install shows before anyone logs in; Admin Settings writes them to the database, and the database wins.

16.2 Edge runtime variables

Server-side only — see Section 8.4 for the full table with requirements and purposes.

16.3 NAS worker variables

VARIABLE	REQUIRED	DEFAULT
`NAS_BASE_PATH`	**Yes**	Survey tree root. Paths outside it are rejected
`NAS_WORKER_TOKEN`	**Non-loopback**	Blank means unauthenticated
`NAS_WORKER_ID`	No	Machine hostname
`STORAGE_CACHE_TTL`	No	30 seconds
`NAS_LOG_JSON`	No	`0`

16.4 Station agent variables

VARIABLE	REQUIRED	DEFAULT

`WATCH_ROOT`	No	Worker base path variable, else blank
`OUT_STAGE`	No	Per station (11.1)
`IN_STAGE`	No	Per station (11.1)
`POINT_MODE`	No	`1` for blur, else `0`
`SUBGRID_PATTERN`	No	`^[A-Za-z]{1,3}[0-9A-Za-z]{2,11}$`
`PROCESS_NAMES`	No	Empty — no process detection
`SCAN_INTERVAL_SEC`	No	`5`
`GROWING_WINDOW_SEC`	No	`90`
`AGENT_TOKEN`	No	Blank means unauthenticated
`AGENT_HOST` / `AGENT_PORT`	No	`0.0.0.0` / `8000`

16.5 NAS worker HTTP surface

METHOD	ROUTE	AUTH	PURPOSE
`GET`	`/health`	**No**	Liveness and the configured mount
`GET`	`/api/folders?path=`	**Yes**	Directory listing with image counts
`GET`	`/api/storage`	**Yes**	Volume usage, per top-level folder
`GET`	`/api/nas-scan?action=`	**Yes**	Survey scans
`GET`	`/api/images/{path}`	**Yes**	Panorama preview with conditional requests

Scan actions: `subgrids`, `survey-folders`, `read-csv`, `folder-images`, `final-images`, `registry`. An unknown action returns **400**.

16.6 Station agent HTTP surface

METHOD	ROUTE	PURPOSE
`GET`	`/health`	Liveness and host telemetry
`GET`	`/api/station`	Full station snapshot: task, output, capture points
`GET`	`/api/station/{id}`	Snapshot, only for the agent's own identifier
`POST`	`/api/rename`	Rename files within a stage folder
`POST`	`/api/sync-bucket`	Start a bucket upload
`GET`	`/api/sync-bucket/{job_id}`	Poll an upload

16.7 Capabilities enforced by the database

16.8 Continuous integration gates

GATE	RUNS
Build	Type check, then the production bundle
Test	Unit and integration suite
Lint	Static analysis — blocking on errors
Python	Compile and unit tests for the worker, on two interpreter versions

There is **no deployment automation** in the repository — the edge platform's Git integration is the deployment mechanism. The station agents and the edge functions are not covered by the automated gates.

Complete and sign once the installation is accepted.

A.1 Installation details

Client	
Site / location	
Supabase project URL	
Web application URL	
NAS worker hostname	
NAS base path	
Station agent hostnames	
Guide revision applied	
Installation date	

A.2 Acceptance checklist

DONE	ITEM	VERIFIED BY	DATE
☐	PostGIS enabled before the first migration		
☐	All 32 migrations applied in order (0011 is a test, run separately)		
☐	Security test script passed (6.2 Tier 1)		
☐	Privilege boundary proven (6.2 Tier 2)		
☐	Panorama bucket created and public		
☐	First administrator bootstrapped and role verified		
☐	Operator accounts created with roles assigned		
☐	NAS folder tree matches Section 9		
☐	NAS worker responding with a valid token		
☐	Four station agents reporting		

- [ ] ~~Five tunnels created and routed over HTTPS~~
- [ ] Tunnels and services set to start automatically
- [ ] Edge project root directory set to the repository root
- [ ] Build-time and runtime variables configured
- [ ] Station addresses registered (12.5)
- [ ] Tier A verification complete (13.1)
- [ ] Tier B verification complete (13.2)
- [ ] Smoke test complete (13.3)
- [ ] Known limitations in Section 14 reviewed with the client

A.3 Signatures

ROLE	NAME	SIGNATURE AND DATE
Client — IT / infrastructure		
Client — survey operations		
Vendor — delivery engineer		

Source documents

This guide consolidates the setup documentation held in the platform repository under `docs/Production Setup/`, together with the worker HTTP contract and the smoke-test checklist. Where this guide and an older note disagree, **this guide is current**.